

# PROJEKT SIECI SPOTKANIE IV

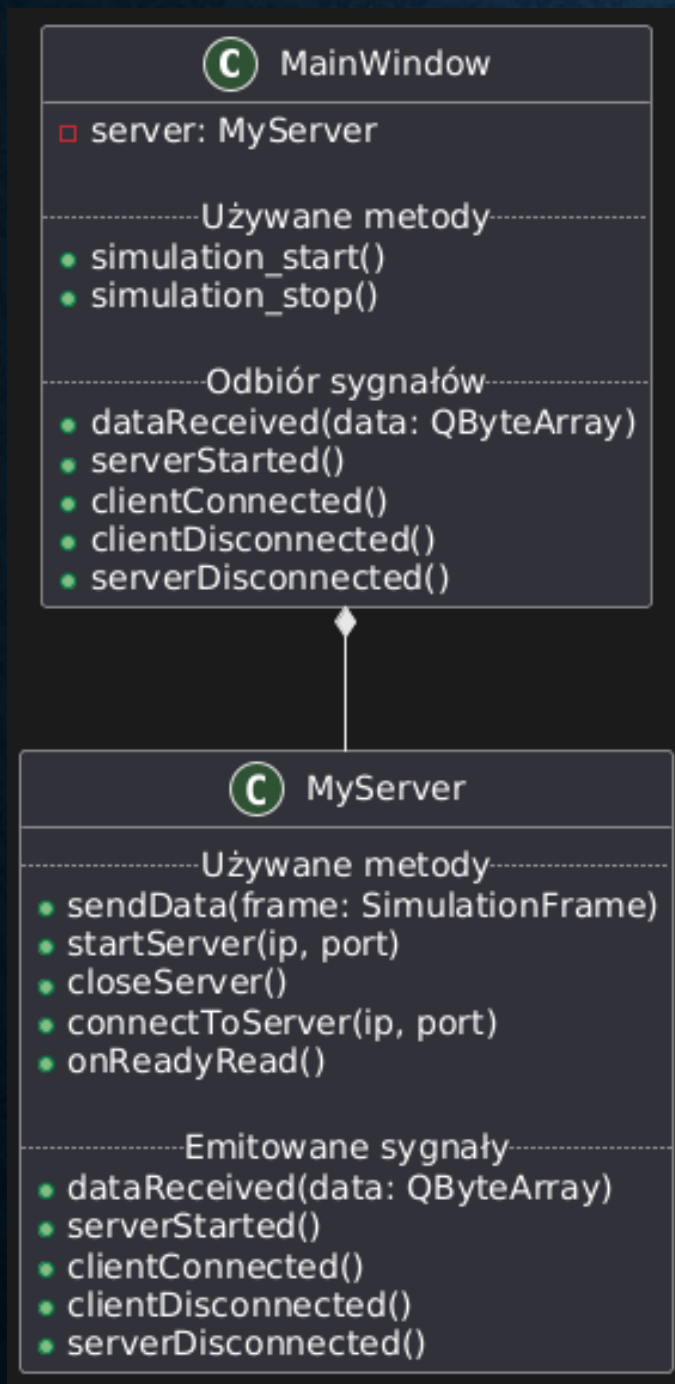
Konrad Szkółka

Michał Witczak



# UML POŁĄCZENIA SIECIOWEGO

- MyServer jest kompozytem MainWindow. Metody są połączone poprzez connect.



# SERIALIZACJA, DESERIALIZACJA I POJEDYNCZA RAMKA

```
SimulationFrame frame{
    .tick = tick,
    .generator_output = generator,
    .current_time = current_time,
    .p = this->pid->proportional_part,
    .i = this->pid->integral_part,
    .d = this->pid->derivative_part,
    .pid_output = pid_output,
    .error = error,
    .arx_output = arx_output,
    .noise = this->arx->noise_part,
};

void MyServer::sendData(SimulationFrame frame)
{
    if (clientSocket && clientSocket->state() == QAbstractSocket::ConnectedState) {
        QByteArray data;
        data.append(reinterpret_cast<const char*>(&frame), sizeof(frame));

        clientSocket->write(data);
        clientSocket->flush();
        qDebug() << "Sent data:" << data;
    } else {
        qWarning() << "No client connected to send data.";
    }
}
```

- Pojedyncza ramka która służy do zwinienia kodu który później przetwarzamy na QByteArray następnie odbierana przez klienta i ponownie przetwarzana.

# OMÓWIENIE GUBIENIA PAKIETÓW

```
if (server->server->isListening() && server->clientSocket) {  
    static auto timer = new QTimer(this);  
  
    timer->setSingleShot(true);  
    connect(timer, &QTimer::timeout, this, [&]() {  
        ui->lamp->setStyleSheet("QLabel { background-color: red; }");  
    });  
  
    timer->start(Simulation::get_instance().get_interval() * 2);  
  
    ui->lamp->setStyleSheet("QLabel { background-color: green; }");  
  
    Simulation::get_instance().arx_output = frame.arx_output;  
    return;  
}
```